

Desarrollo de una Aplicación Ionic + Angular (Standalone) + Capacitor + GitHub + Vercel

Objetivo del proyecto

Construir una aplicación completa utilizando:

- Ionic
- Angular Standalone
- TypeScript
- Capacitor
- Android Studio
- GitHub
- Vercel

La aplicación dispondrá de:

Home

Menú horizontal de navegación hacia:

- Productos
- Nosotros
- Contacto

Productos

- Consumirá datos desde un fichero JSON local
- Mostrará:
 - id
 - nombre
 - unidades
 - precio
 - fotografía

Nosotros

- Descripción de la empresa
- Obtención de la geolocalización actual del dispositivo
- Cálculo de distancia respecto a:

Calle Arcos de Jalón 18
Madrid

Contacto

Formulario:

- Correo electrónico
- Mensaje

Persistencia local mediante almacenamiento de mensajes.

Despliegue

- GitHub
- Vercel

Ejecución móvil

- Capacitor
- Android Studio
- Emulador Pixel 9

1. Creación del proyecto

Generación

ionic start corporateApp blank --type=angular --standalone

Acceder al directorio:

cd corporateApp

Ejecutar:

ionic serve

2. Estructura inicial

Crearemos la siguiente organización:

```
src
|
├── app
|   ├── pages
|   |   ├── home
|   |   ├── productos
|   |   ├── nosotros
|   |   └── contacto
|   |
|   ├── services
|   |   ├── products.service.ts
|   |   ├── geolocation.service.ts
|   |   └── messages.service.ts
|   |
|   └── models
|       └── product.interface.ts
|
└── assets
    ├── data
    |   └── products.json
    └── images
```

3. Generación de páginas

```
ionic generate page pages/home --standalone
```

```
ionic generate page pages/productos --standalone
```

```
ionic generate page pages/nosotros --standalone
```

```
ionic generate page pages/contacto --standalone
```

4. Configuración de rutas

Archivo:

app.routes.ts

```
import { Routes } from '@angular/router';
```

```
export const routes: Routes = [
```

```
{
  path: "",
  redirectTo: 'home',
  pathMatch: 'full'
},

{
  path: 'home',
  loadComponent: () =>
  import('./pages/home/home.page')
  .then(m => m.HomePage)
},

{
  path: 'productos',
  loadComponent: () =>
  import('./pages/productos/productos.page')
  .then(m => m.ProductosPage)
},

{
  path: 'nosotros',
```

```
loadComponent: () =>
import('./pages/nosotros/nosotros.page')
.then(m => m.NosotrosPage)
},

{
  path: 'contacto',
  loadComponent: () =>
import('./pages/contacto/contacto.page')
.then(m => m.ContactoPage)
}

];
```

5. Menú principal horizontal

Home

```
<ion-header>
  <ion-toolbar>
    <ion-title>Corporate App</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content>

  <ion-segment>

    <ion-segment-button routerLink="/productos">
      <ion-label>Productos</ion-label>
    </ion-segment-button>

    <ion-segment-button routerLink="/nosotros">
      <ion-label>Nosotros</ion-label>
    </ion-segment-button>

    <ion-segment-button routerLink="/contacto">
      <ion-label>Contacto</ion-label>
    </ion-segment-button>

  </ion-segment>

</ion-content>
```

Corporate App Productos Nosotros Contacto

6. Modelo de producto

```
export interface Product {  
  
  id: number;  
  
  nombre: string;  
  
  unidades: number;  
  
  precio: number;  
  
  foto: string;  
}
```

7. Datos fake (JSON local)

Archivo:

assets/data/products.json

```
[  
  {  
    "id": 1,  
    "nombre": "Portátil Dell",  
    "unidades": 12,  
    "precio": 1200,  
    "foto": "assets/images/dell.jpg"  
  },  
  {  
    "id": 2,  
    "nombre": "Monitor LG",  
    "unidades": 8,  
    "precio": 299,  
    "foto": "assets/images/lg.jpg"  
  }  
]
```

8. Servicio para productos

Generación:

```
ionic generate service services/products
```

products.service.ts

```
import { Injectable } from '@angular/core';
import { Product } from '../models/product.interface';
```

```
@Injectable({
  providedIn: 'root'
})
export class ProductsService {

  async getProducts(): Promise<Product[]> {

    const response =
      await fetch('assets/data/products.json');

    const products =
      await response.json();

    return products;
  }
}
```

9. Componente Productos

productos.page.ts

```
import { Component, OnInit } from '@angular/core';
import { ProductService } from '../services/products.service';
```

```
@Component({
  standalone: true,
  selector: 'app-productos',
  templateUrl: './productos.page.html'
})
export class ProductosPage implements OnInit {

  products: any[] = [];

  constructor(
    private productService: ProductService
  ) {}

  async ngOnInit() {

    this.products =
      await this.productService.getProducts();

  }

}
```

productos.page.html

<ion-content>

<ion-grid>

<ion-row class="header">

<ion-col>ID</ion-col>

<ion-col>Nombre</ion-col>

<ion-col>Unidades</ion-col>

<ion-col>Precio</ion-col>

<ion-col>Foto</ion-col>

</ion-row>

<ion-row *ngFor="let product of products">

<ion-col>{{product.id}}</ion-col>

<ion-col>{{product.nombre}}</ion-col>

<ion-col>{{product.unidades}}</ion-col>

<ion-col>{{product.precio}} €</ion-col>

<ion-col>

</ion-col>

</ion-row>

</ion-grid>

</ion-content>

ID Nombre Unidades Precio Foto {{product.id}} {{product.nombre}} {{product.unidades}} {{product.precio}} €

10. Geolocalización

Instalación:

```
npm install @capacitor/geolocation
```

```
npx cap sync
```

11. Servicio geográfico

```
import { Injectable } from '@angular/core';  
import { Geolocation } from '@capacitor/geolocation';
```

```
@Injectable({  
  providedIn: 'root'  
})  
export class GeolocationService {  
  
  async getCurrentPosition() {  
  
    const data =  
      await Geolocation.getCurrentPosition();  
  
    return data.coords;  
  }  
}
```

12. Coordenadas oficinas

Arcos de Jalón 18 (Madrid)

A efectos de la práctica:

```
const oficinaLat = 40.4452;  
const oficinaLon = -3.6115;
```

13. Fórmula Haversine

Permite calcular distancia real sobre la superficie terrestre.

```
calcularDistancia(  
  lat1:number,  
  lon1:number,  
  lat2:number,  
  lon2:number  
)  
{  
  
  const R = 6371;  
  
  const dLat =  
    (lat2-lat1) * Math.PI/180;  
  
  const dLon =  
    (lon2-lon1) * Math.PI/180;  
  
  const a =  
    Math.sin(dLat/2)  
    Math.sin(dLat/2) +  
    Math.cos(lat1Math.PI/180)  
    Math.cos(lat2Math.PI/180) *  
    Math.sin(dLon/2)  
    Math.sin(dLon/2);  
  
  const c =  
    2 * Math.atan2(  
      Math.sqrt(a),  
      Math.sqrt(1-a)  
    );  
  
  return Rc;  
}
```

14. Nosotros

Obtendrá:

- ubicación del usuario
- distancia respecto a la oficina

Mostrando:

```
<ion-card>
```

```
  <ion-card-header>
```

```
    <ion-card-title>
```

```
    Nuestra empresa
```

```
    </ion-card-title>
```

```
  </ion-card-header>
```

```
  <ion-card-content>
```

```
    Empresa especializada en  
    soluciones basadas en Angular,  
    Ionic y Node.js.
```

```
  <p>
```

```
    Distancia a nuestras oficinas:
```

```
    {{ distancia }} km
```

```
  </p>
```

```
</ion-card-content>
```

```
</ion-card>
```

```
``
```

Nuestra empresa Empresa especializada en soluciones basadas en Angular, Ionic y Node.js.

Distancia a nuestras oficinas: {{ distancia }} km

```
``
```

15. Contacto

Forms

```
correo = "";
```

```
mensaje = "";
```

```
enviado = false;
```

HTML

```
<ion-item>
```

```
  <ion-input  
    [(ngModel)]="correo"  
    placeholder="Correo">  
  </ion-input>
```

```
</ion-item>
```

```
<ion-item>
```

```
  <ion-textarea  
    [(ngModel)]="mensaje">  
  </ion-textarea>
```

```
</ion-item>
```

```
<ion-button  
(click)="enviar()">
```

Enviar

```
</ion-button>
```

16. Persistencia local

En navegador no es posible escribir directamente un TXT.

Para la práctica utilizaremos:

@capacitor/preferences

Instalación:

```
npm install @capacitor/preferences
```

Guardar mensaje:

```
await Preferences.set({  
  key: 'ultimoMensaje',  
  value: JSON.stringify({  
    correo:this.correo,  
    mensaje:this.mensaje  
  })  
});
```

Cuando la aplicación se ejecute en Android puede evolucionarse posteriormente a:

Filesystem

SQLite

Backend Node

17. GitHub

Inicializar repositorio:

```
git init
```

Agregar archivos:

```
git add .
```

Commit:

```
git commit -m "Proyecto Ionic corporativo"
```

Crear repositorio en GitHub y conectar:

```
git remote add origin https://github.com/usuario/corporateApp.git
```

Push:

```
git branch -M main
```

```
git push -u origin main
```

18. Deploy automático en Vercel

Importar repositorio GitHub

Entrar en:

<https://vercel.com>

Seleccionar:

Add New Project

Elegir:

Repositorio corporateApp

Configuración

Framework:

Other

..

Build:

npm run build

Output:

www

..

Cada push en GitHub generará:

Deploy automático

19. Android

Añadir Android:

```
npx cap add android
```

Sincronizar:

```
ionic build
```

```
npx cap sync
```

Abrir Android Studio:

```
npx cap open android
```

20. Emulador Pixel 9

Crear dispositivo virtual:

Device Manager

Seleccionar:

Pixel 9

API recomendada:

Android 15

Ejecutar:

Run App

Aspectos técnicos

Standalone Components

Angular 20 utiliza componentes standalone como estrategia principal de modularización.

Ventajas:

- Menos complejidad
- Menos módulos
- Mejor tree shaking
- Menor tamaño final

Lazy Loading

Todas las páginas se cargan bajo demanda mediante:

`loadComponent()`

Esto genera chunks independientes optimizando el rendimiento.

Capacitor

Permite acceso a:

- GPS
- Cámara
- Filesystem
- Bluetooth
- Notificaciones
- Sensores

usando TypeScript y JavaScript.

Arquitectura utilizada

Pages

- |— Home
- |— Productos
- |— Nosotros
- └— Contacto

Services

- |— ProductService
- |— GeolocationService
- └— MessageService

Assets

- |— JSON
- └— Imágenes

Esta misma arquitectura es perfectamente escalable para sustituir posteriormente:

JSON Local



API REST Node.js



Base de datos SQL / MongoDB

sin modificar la estructura principal de la aplicación.